

sumdoku

April 26, 2025

- 1 Killer Sudoku Solver
- 2 Project 3, DSA-8420
- 3 By: Saransh Rakshak
- 4 Due: April 29, 2025



4.1 Killer Sudoku Solver With Pyomo

This notebook solves *killer sudoku* puzzles using Pyomo for mathematical optimization. The code supports arbitrary puzzle inputs and includes step by step instructions to modify and solve custom puzzles. To run the notebook please press **Shift+Enter** on each Python cell (code block), or click **Run All** to solve predefined killer sudoku puzzle. Please do not modify any details outside of “**Step 2 (User Input)**” !

The rules of **killer sudoku** are defined as follows:

1. Fill all rows, columns, and 3x3 blocks with numbers 1-9 exactly like in classic sudoku.
2. Pay attention to the cages – groups of cells indicated by dotted lines.
3. Make sure the sum of numbers in each cage is equal to the number in the upper left corner of the cage.
4. Numbers cannot repeat within cages, a single row, column, or 3x3 region.

To solve your own custom killer sudoku puzzle, modify the values in **Step 2 (User Input)** with your killer sudoku setup.

4.1.1 Do not modify any values or code in any step besides “Step 2 (User Input)”.

4.2 Step 1: Importing Libraries

Please import the following libraries by running the code cell (**Shift+Enter**). If not already installed, please install using **pip**

```
pip install numpy pyomo glpk
```

or **conda**

```
conda install -c conda-forge numpy pyomo glpk
```

in your system's command line.

```
[1]: # Import Python Libraries
import numpy as np
import pyomo.environ as pyo
from pyomo.opt import SolverFactory
```

4.3 Step 2 (User Input): Initializing the Puzzle

Modify variables *known*, *cages*, and *cage_sums* to match custom killer sudoku puzzle.

- *known*: list of fixed cells, in format [(row, col, digit)].
- *cages*: list of cage groups with cell coordinates.
- *cage_sums*: list of cage totals matching *cages* order.

To solve custom killer sudoku puzzle, input own values to variables *known*, *cages* and *cage_sums*.

Run the following code cell to define killer sudoku puzzle initial setup.

Below is an example of a killer sudoku puzzle being performed. Its associated values will be used for the example.

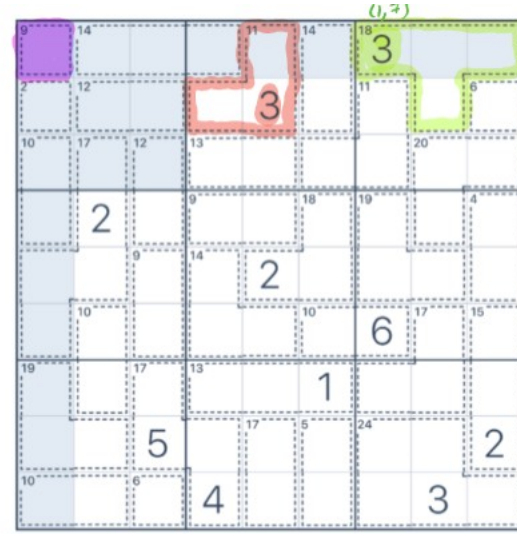


Figure 1: From: <https://sudoku.com>

Below is an example format for representing the input data. Feel free to adjust the structure as needed to best suit your implementation.

known = [(1, 7, 3), (2, 5, 3), (4, 2, 2), (5, 5, 2), (6, 7, 6), (7, 6, 1), (8, 3, 5), (8, 9, 2), (9, 4, 4), (9, 8, 3)]

cage = [(1,1), [(1,2), (1,3), (1,4)], [(1,5), (2,4), (2,5)], [(1,6), (2,6)], [(1,7), (1,8), (1,9), (2,8)], [(2,1)], [(2,2), (2,3)], [(2,7), (3,6), (3,7)], [(2,9)], [(3,1), (4,1)], [(3,2), (4,2), (5,1), (5,2), (6,1)], [(3,3), (4,3)], [(3,4), (3,5)], [(3,8), (3,9), (4,8)], [(4,4), (4,5)], [(4,6), (5,5), (5,6)], [(4,7), (5,7), (5,8)], [(4,9), (5,9)], [(5,3), (6,3)], [(5,4), (6,4), (6,5)], [(6,2), (7,2)], [(6,6), (6,7)], [(6,8), (7,7), (7,8)], [(6,9), (7,9), (8,9)], [(7,1), (8,1), (8,2)], [(7,3), (8,3), (8,4), (9,4)], [(7,4), (7,5), (7,6)], [(8,5), (9,5)], [(8,6), (9,6)], [(8,7), (8,8), (9,7), (9,8), (9,9)], [(9,1), (9,2)], [(9,3)]

cage_sum = [9, 14, 11, 14, 18, 2, 12, 11, 6, 10, 17, 12, 13, 20, 9, 18, 19, 4, 9, 14, 10, 10, 17, 15, 19, 17, 13, 17, 5, 24, 10, 6]

```
[2]: """
known: list fixed cells.
known = [(<row>, <col>, <dig>), (<i>, <j>, <k>), ... , (<row>, <col>, <dig>)].

Replace example values with custom values for personal killer sudoku solution.
"""
known = [(1, 7, 3), (2, 5, 3), (4, 2, 2), (5, 5, 2),
         (6, 7, 6), (7, 6, 1), (8, 3, 5), (8, 9, 2),
         (9, 4, 4), (9, 8, 3)]
```

```

"""
cages: list cage group with each cell coordinate.
cages = [[<Cage_Groups>], [(<i>,<j>), (<i>,<j>)], ... , [<Cage_Groups>]].

Replace example values with custom values for personal killer sudoku solution.
"""
cages = [[(1,1)], [(1,2), (1,3), (1,4)], [(1,5), (2,4), (2,5)],
          [(1,6), (2,6)], [(1,7), (1,8), (1,9), (2,8)], [(2,1)],
          [(2,2), (2,3)], [(2,7), (3,6), (3,7)], [(2,9)],
          [(3,1), (4,1)], [(3,2), (4,2), (5,1), (5,2), (6,1)],
          [(3,3), (4,3)], [(3,4), (3,5)], [(3,8), (3,9), (4,8)],
          [(4,4), (4,5)], [(4,6), (5,5), (5,6)], [(4,7), (5,7), (5,8)],
          [(4,9), (5,9)], [(5,3), (6,3)], [(5,4), (6,4), (6,5)],
          [(6,2), (7,2)], [(6,6), (6,7)], [(6,8), (7,7), (7,8)],
          [(6,9), (7,9), (8,9)], [(7,1), (8,1), (8,2)],
          [(7,3), (8,3), (8,4), (9,4)], [(7,4), (7,5), (7,6)],
          [(8,5), (9,5)], [(8,6), (9,6)],
          [(8,7), (8,8), (9,7), (9,8), (9,9)],
          [(9,1), (9,2)], [(9,3)]]

"""
cage_sums: list of cage totals, must match cages order.
cage_sums = [<Int>, ... , <Integer>].

Replace example values with custom values for personal killer sudoku solution.
"""
cage_sums = [9, 14, 11, 14, 18, 2, 12, 11, 6, 10, 17, 12, 13, 20, 9,
             18, 19, 4, 9, 14, 10, 10, 17, 15, 19, 17, 13, 17, 5, 24, 10, 6]

```

4.4 Step 3: Creating Model and Variables

Run the following cell to create our puzzle solver with a Pyomo concrete model.

```

[3]: # pyomo model
model = pyo.ConcreteModel()

# sudoku grid
N = pyo.RangeSet(1,9)
# 3by3 block ind
B = pyo.RangeSet(1,3)

# x[i,j,k]= 1 if i,j == k
model.x = pyo.Var(N,N,N, within= pyo.Binary)
# y[i,j] = digit in cell i,j
model.y = pyo.Var(N, N, within= pyo.Integers, bounds= (1,9))

```

4.5 Step 4: Defining Objective Function

Run the following cell to establish an objective function to our Pyomo concrete model. The following setup prepares the model for finding a working solution to the puzzle.

```
[4]: # no optimizing goal just finding solution
model.obj = pyo.Objective(expr= 0)
```

4.6 Step 5: Establishing Sudoku Constraints

Run the following code cell to establish rules (constraints) of sudoku for our solver.

```
[ ]: # one digit per cell
model.cell_unique = pyo.ConstraintList()
for i in N:
    for j in N:
        model.cell_unique.add(sum(model.x[i,j,k] for k in N)==1)

# one digit occurrence in row
model.row_unique = pyo.ConstraintList()
for i in N:
    for k in N:
        model.row_unique.add(sum(model.x[i,j,k] for j in N)==1)

# one digit occurrence in col
model.col_unique = pyo.ConstraintList()
for j in N:
    for k in N:
        model.col_unique.add(sum(model.x[i,j,k] for i in N)==1)

# one digit occurrence in 3 by 3
model.block_unique = pyo.ConstraintList()
for block_row in range(0,9,3):
    for block_col in range(0,9,3):
        for k in N:
            model.block_unique.add(
                sum(model.x[i+1, j+1, k]
                    for i in range(block_row, block_row + 3)
                    for j in range(block_col, block_col + 3)
                ) == 1)
```

4.7 Step 6: Link x , y Variables

Refer to cell's value directly with $y[i,j]$.

```
[6]: # y[i,j]=k only if x[i,j,k]==1
model.link = pyo.ConstraintList()
for i in N:
    for j in N:
```

```
model.link.add(model.y[i,j] == sum(k * model.x[i,j,k] for k in N))
```

4.8 Step 7: Establishing Known Values

Run the following code cell to assign your entered values for the killer sudoku puzzle set up.

```
[7]: model.known = pyo.ConstraintList()
      for i, j, k in known:
          model.known.add(model.x[i,j,k] == 1)
```

4.9 Step 8: Integrate Cage Constraints

Run the following code cell to incorporate killer sudoku cage constraint rules.

```
[8]: # sum of num in each cage == upper left corner of cage
model.this_cage_sum = pyo.ConstraintList()
for ind, cage in enumerate(cages):
    model.this_cage_sum.add(sum(model.y[i,j] for i,j in cage) == cage_sums[ind])

# cant repeat within cages
model.cage_unique = pyo.ConstraintList()
for cage in cages:
    for k in N:
        model.cage_unique.add(sum(model.x[i,j,k] for i,j in cage) <= 1)
```

4.10 Step 9: Solve Puzzle

Run the following code cell to use GLPK/Pyomo to determine a valid killer sudoku solution based on our defined values and rules.

```
[9]: # solving with glpk
solver = SolverFactory('glpk')
result = solver.solve(model, tee=True)
```

GLPSOL--GLPK LP/MIP Solver 5.0

Parameter(s) specified in the command line:

--write C:\Users\sar-home\AppData\Local\Temp\tmp9ij_xpsw.glpk.raw --wglp

C:\Users\sar-home\AppData\Local\Temp\tmp91fzd5mq.glpk.glp --cpxlp C:\Users\sar-home\AppData\Local\Temp\tmptw7u_15e.pyomo.lp

Reading problem data from 'C:\Users\sar-home\AppData\Local\Temp\tmptw7u_15e.pyomo.lp'...

C:\Users\sar-home\AppData\Local\Temp\tmptw7u_15e.pyomo.lp:7655: warning: lower bound of variable 'x4' redefined

C:\Users\sar-home\AppData\Local\Temp\tmptw7u_15e.pyomo.lp:7655: warning: upper bound of variable 'x4' redefined

735 rows, 811 columns, 4546 non-zeros

810 integer variables, 729 of which are binary

8384 lines were read

Writing problem data to 'C:\Users\sar-

```

home\AppData\Local\Temp\tmp91fzd5mq.glpk.glp'...
6831 lines were written
GLPK Integer Optimizer 5.0
735 rows, 811 columns, 4546 non-zeros
810 integer variables, 729 of which are binary
Preprocessing...
504 rows, 452 columns, 2417 non-zeros
452 integer variables, 389 of which are binary
Scaling...
  A: min|aij| = 1.000e+00 max|aij| = 9.000e+00 ratio = 9.000e+00
Problem data seem to be well scaled
Constructing initial basis...
Size of triangular part is 400
Solving LP relaxation...
GLPK Simplex Optimizer 5.0
504 rows, 452 columns, 2417 non-zeros
   0: obj = 0.000000000e+00 inf = 4.731e+02 (197)
  353: obj = 0.000000000e+00 inf = 6.721e-13 (0) 3
OPTIMAL LP SOLUTION FOUND
Integer optimization begins...
Long-step dual simplex will be used
+ 353: mip = not found yet >= -inf (1; 0)
+ 353: >>>> 0.000000000e+00 >= 0.000000000e+00 0.0% (1; 0)
+ 353: mip = 0.000000000e+00 >= tree is empty 0.0% (0; 1)
INTEGER OPTIMAL SOLUTION FOUND
Time used: 0.0 secs
Memory used: 1.6 Mb (1634612 bytes)
Writing MIP solution to 'C:\Users\sar-
home\AppData\Local\Temp\tmp9ij_xpsw.glpk.raw'...
1555 lines were written

```

4.11 Step 10: Display Solution

Run the following code cell to print out arranged killer sudoku values.

```

[10]: # pretty display of solution
sol = np.zeros((9,9), dtype= int)
for i in N:
    for j in N:
        sol[i-1, j-1]= int(pyo.value(model.y[i,j]))

print("Solved Killer Sudoku")
print("=====")
print(sol)

```

```

Solved Killer Sudoku
=====
[[9 8 4 2 7 6 3 1 5]

```

```
[2 5 7 1 3 8 4 9 6]
[6 1 3 9 4 5 2 7 8]
[4 2 9 3 6 7 8 5 1]
[5 6 1 8 2 9 7 4 3]
[3 7 8 5 1 4 6 2 9]
[8 3 2 7 5 1 9 6 4]
[7 4 5 6 9 3 1 8 2]
[1 9 6 4 8 2 5 3 7]]
```